

Question 1: BWT Encoding 1a. In the language of your choice, implement a BWT encoder and encode the string below. Faster (Linear time) methods exist for computing the BWT, although for this assignment you can use the simple method based on standard sorting techniques. Your solution does not need to be an optimal algorithm and can use $O(n^2)$ space and $O(n^2 \lg n)$ time.

```
def computeBWT(s):
    ## add the magic end-of-string character
    s = s + "$"

    ## build up the BWM from the cyclic permutations
    ## note the ith cyclic permutation is just "s[i..n] + s[0..i]"
    rows = []
    for i in range(len(s)):
        rows.append(s[i:len(s)] + s[:i])

    ## just use the builtin sort command to sort the cyclic permutations
    rows = sorted(rows)

    ## now extract the last column
    bwt = ""
    for k in range(len(s)):
        bwt += rows[k][len(s)-1:len(s)]

    return bwt

text = """ I_am_fully_convinced_that_species_are_not_immutable;_but_that_those_belonging_to_what_are_

encoded_string = computeBWT(text)

print(encoded_string)

.$ etsense__ .,eIlrftassrse;;emyleeymedntt,ee,fetetssssyeelftttedfdtsetndntgdort_ercr__ss_metdd_____
```

1b. The original motivation for the BWT was for data compression, especially through a run-length encoding where repeats ("runs") of a character are replaced by a count. For example the string "GGGGAAAAATTTTTTACCCCCCCCCAAAAAAA" (30 characters) can be encoded as "G4A5T6AC8A7" (11 characters) (36% of the original length). Implement a runlength encoding algorithm and compute 1) the length of the original string from 1a, 2) the length of a runlength encoded version of the same string, and 3) the length of the runlength encoded version of the BWT of that same string

```
def runlen_encode(string):
    i = 0
    str_arr = []

    while i < len(string):
        curr = string[i]

        count = 1
        offset = i + 1
```

```

        while offset < len(string) and string[offset] == curr:
            count += 1
            offset += 1

        str_arr.append(f"{curr}{count if count != 1 else ''}")
        i += count

    return ''.join(str_arr)

orig_len = len(text)

orig_runlen_encoded = runlen_encode(text)
orig_runlen = len(orig_runlen_encoded)

BWT_runlen_encoded = runlen_encode(encoded_string)
BWT_runlen = len(BWT_runlen_encoded)

print(f"The original string was {orig_len} chars long.\nThe original string with runlength encoding was {orig_runlen} chars long.\nBWT string with runlength encoding was {BWT_runlen} chars long.")

```

The original string was 413 chars long.
The original string with runlength encoding was 413 chars long.
The BWT string with runlength encoding was 355 chars long.

1c. Now compute the effectiveness of a runlength encoding and the BWT on a 10kb segment of human chromosome 22. Make sure to report 1) the length of the original string, 2) the length of the runlength encoded version of the same string, and 3) the length of the runlength encoded version of the BWT of that string. Before running `rle` or `bwt`, clean the fasta file by replacing lowercase acgt with ACGT, replacing any other characters with N, removing the fasta header, and removing all the newlines (the genome sequence should appear by itself on one very long line of text)

```

#clean_fasta.py

import sys

#Store both input and output file names (CLI)
input_filename = sys.argv[1]
output_filename = sys.argv[2]

#Open sequence
input_file = open(input_filename)

file_clean = open(output_filename, 'w')

input_file.readline() #throw out the first line containing the name of the file

for line in input_file:

    line = line.strip().upper()

```

```

clean_line = []

for c in line:
    if c in ['A', 'C', 'G', 'T']:
        clean_line.append(c)
    else:
        clean_line.append('N')

file_clean.write(''.join(clean_line))

input_file.close()
file_clean.close()

$ wget https://github.com/schatzlab/appliedgenomics2026/raw/refs/heads/main/assignments/assignment1/chr22.fa.gz
$ gunzip chr22.fa.gz
$ samtools faidx chr22.fa
$ samtools faidx chr22.fa chr22:20000000-20010000 > chr22_orig.fa

$ python clean_fasta.py chr22_orig.fa chr22.clean
$ compute_length.py chr22.clean

$ rle.py chr22.clean > chr22.clean.rle
$ compute_length.py chr22.clean.rle

$ bwt.py chr22.clean > chr22.bwt
$ rle.py chr22.bwt > chr22.bwt.rle
$ compute_length.py chr22.bwt.rle

file = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment3/chr22.clean")
sequence = file.readline()

print(f"The length of the original string is {len(sequence)}")

chr22_runlen = runlen_encode(sequence)

print(f"The length of the runlength-encoded string is {len(chr22_runlen)} chars long.")

chr22_BWT_runlen = runlen_encode(computeBWT(sequence))

print(f"The length of the runlength-encoded version of the BWT of the string is {len(chr22_BWT_runlen)} chars long.")

The length of the original string is 10001
The length of the runlength-encoded string is 8966 chars long.
The length of the runlength-encoded version of the BWT of the string is 8862 chars long.

```

Questions 2: Binomial Distribution 2a. For coverage $n = 10$ to 200 , calculate the maximum number of minor allele reads (round down) that would make your one-sided binomial test reject the null hypothesis $p=0.5$ at 0.05 significance. Plot coverage on the x-axis and $(\text{number of reads})/(\text{coverage})$ on the y-axis. Note this is the minimum number of reads that are necessary to believe we might have

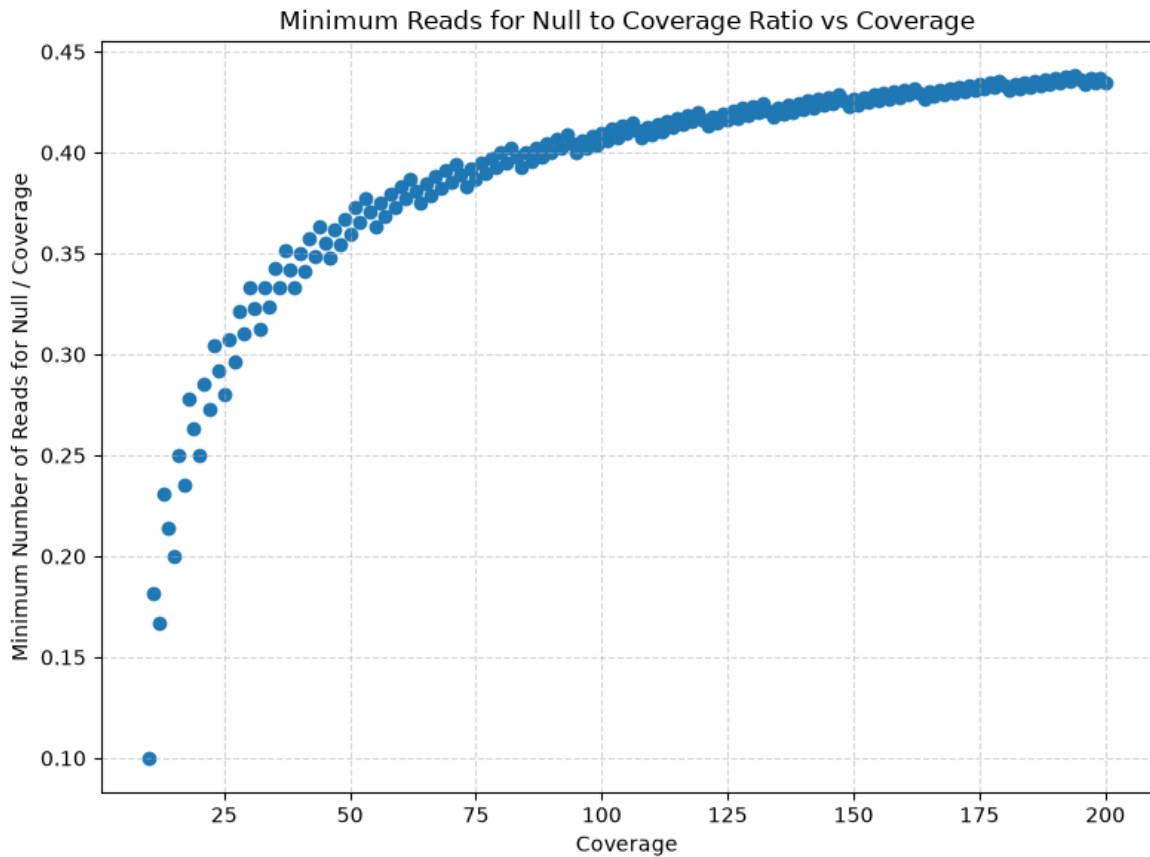
a heterozygous variant on the second haplotype rather than just mere sequencing error.

```
import numpy as np
from scipy.stats import binom
import matplotlib.pyplot as plt

coverage = np.array(range(10, 201))
max_num = []

max_num = binom.ppf(0.05, coverage, 0.5) - 1
ratio = max_num / coverage

plt.figure(figsize = (8, 6))
plt.scatter(coverage, ratio)
plt.xlabel("Coverage")
plt.ylabel("Minimum Number of Reads for Null / Coverage")
plt.title('Minimum Reads for Null to Coverage Ratio vs Coverage')
plt.grid(True, which='major', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



2b. What asymptote does the plot seem to approach? Why is this?

Answer: The plot seems to approach an asymptotic minimum read number to coverage ratio of 0.5. This is because as coverage increases, the number of observations also increases, so it's easier to view deviations from the expected 50% incidence rate. From a mathematical perspective, the minimum number of minor allele reads is dependent on the expected number, $0.5n$, and standard deviation, which scales with $\sqrt{n}$ where n is the coverage. Thus as coverage increases, so does the difference between the minimum number and the expected number. However, when the minimum number is divided by the coverage, n , we get $\sqrt{n}$ in the denominator instead, which pushes the term to 0. This causes $0.5n$ to be the surviving term, which is why the minor allele count to coverage ratio approaches 0.5 asymptotically.

Question 3: Identifying a Specific Variant Three friends, Ezra, Sabine, and Zeb go out to lunch together and, in discussing the menu, all realize that they all have a specific culinary preference. They believe the cause of this preference to be genetic. Can you identify what SNP causes this preference and what the preference is?

3a. You are able to narrow down the SNP to a specific region in chromosome 11. Using bowtie2, align their FASTQ files to the provided reference file. How many reads does each file have and how many are successfully mapped? [Hint: Use samtools flagstat.]

```
Build a bowtie2 index (BWT)
$ bowtie2-build ref.fa ref
```

```
Now align reads, sort, and compute alignment stats
$ bowtie2 -x ref -1 PREFIX.1.fq -2 PREFIX.2.fq -S PREFIX.sam
$ samtools sort PREFIX.sam -o PREFIX.bam
$ samtools flagstat PREFIX.bam > PREFIX.flagstat
```

Answer: Ezra has 902,256 reads and 855,264 of them were successfully mapped. Sabine had 902,256 reads and 855,033 were successfully mapped. Zeb had 902,256 reads and 854,641 were successfully mapped.

3b. How many high confidence SNPs and indels does each file have? [Hint: Sort the SAM file first. Then, call variants with freebayes. Then filter for the variant quality to be at least 30 and normalize using bcftools. Summarize using bcftools stats.]

```
## run freebayes of the sorted alignments against the reference
$ freebayes -f ref.fa PREFIX.bam > PREFIX.vcf

## filter, normalize, and compute stats
$ bcftools view -i 'QUAL>=30' PREFIX.vcf -Oz -o PREFIX.filtered.vcf
$ bcftools norm -f ref.fa PREFIX.filtered.vcf > PREFIX.norm.vcf
$ bcftools stats PREFIX.norm.vcf > PREFIX.norm.vcf.stats

## Compress and index the variants
$ bgzip PREFIX.norm.vcf
$ bcftools index PREFIX.norm.vcf.gz

## Now compare the variants in the three samples
$ bcftools isec PREFIX1.norm.vcf.gz PREFIX2.norm.vcf.gz PREFIX3.norm.vcf.gz -p norm -n=3
```

Answer: Sorted using samtools as stated in previous question. Used freebayes and bcftools commands above with PREFIX replaced with "ezra", "zeb", and "sabine". Ezra has 1671 high-confidence SNPs and 181 high-confidence indels. Sabine has 1843 high-confidence SNPs and 160 high-confidence indels. Zeb has 1694 high-confidence SNPs and 163 high-confidence indels.

3c. You now know which SNPs and indels each friend has. However, you want to know which variant they all share. How many variants are shared between all three friends? [Hint: Use bcftools isec after filtering and normalizing the variants.]

There are 9 shared variants between all three friends. The output of bcftools isec is pasted below:

```
chr11:5868417-7868417 1000000 C A 111
chr11:5868417-7868417 1080843 T A 111
chr11:5868417-7868417 1080953 A G 111
chr11:5868417-7868417 1080967 G A 111
chr11:5868417-7868417 1081061 G T 111
chr11:5868417-7868417 1158147 A T 111
chr11:5868417-7868417 1158491 G A 111
chr11:5868417-7868417 1158511 TTA ATG 111
chr11:5868417-7868417 1158521 A G 111
```

3d. Between the variants shared between all three friends, which is likeliest to cause a phenotype of interest? [Hint: The variant should be homozygous in all 3 samples and will be in a gene that has a function related to taste. You can search for variants at a certain chromosome and position at <https://genome.ucsc.edu/cgi-bin/hgTracks?db=hg38>. Remember, the position in the intersected VCF is the position within the region we're looking at, so you will have to find the starting location of the region by shifting over by 5868417! Use `bcftools view -i 'GT="1/1"' PREFIX.vcf` to filter for the correct genotype.]

Filtering for homozygous:

```
bcftools view -i 'GT="1/1"' 0000.vcf > hom0.vcf
bcftools view -i 'GT="1/1"' 0001.vcf > hom1.vcf
bcftools view -i 'GT="1/1"' 0002.vcf > hom2.vcf
```

Find shared:

```
bgzip hom*
bcftools index hom0.vcf.gz
bcftools index hom1.vcf.gz
bcftools index hom2.vcf.gz
bcftools isec hom0.vcf.gz hom1.vcf.gz hom2.vcf.gz -p hom_shared -n=3
```

Running these commands, we find that there is one variant shared between all three individuals that fits the homozygous criteria:

```
chr11:5868417-7868417 1000000 C A 111
```

This must be the most likely to cause a phenotype of interest.

3e. What is the phenotype? [Hint: Search the name of the gene associated with the variant you found in 3d.]

Answer: The variant appears to be in a gene called OR10A2, which encodes an olfactory receptor. In individuals with the C to A mutation, olfactory receptors bind to aldehydes which are found in both cilantro and commercial soaps. As a result, people with this SNV dislike the taste of cilantro since it tastes like soap to them. This would explain the shared culinary preference of the trio. They all likely hate cilantro.

Question 4: Planetary Population Analysis You have synthetic genotypes from five populations: mercury, venus, mars, saturn, and pluto. Explore their genetic relationships using PCA, t-SNE, and UMAP, then train a classifier to predict the population of origin of additional unlabeled people. Infer which populations are most similar from your analysis.

Each person has 100 binary variant values (0 or 1). These are simplified allele indicators, rather than diploid 0/1/2 allele dosages. Rows are shuffled, and the challenge population labels are hidden. Sample IDs identify people and should not be used as genetic features.

Use Python with pandas, NumPy, matplotlib, scikit-learn, and umap-learn (see the resources below). A Jupyter notebook is recommended. Each part should require only about 20 lines of code; you can reuse a small plotting helper. Use `random_state=2026` for stochastic methods and the validation split, and keep population colors consistent across plots.

4a. Load the files with `pandas.read_csv`. Build a feature matrix from the labeled data by creating a dataframe called `X` that contains the `sample_id` and 100 variant columns. Build a label vector

by creating a dataframe called `y` that contains only the population column. Report the two feature matrix shapes, confirm that the genotypes contain only zeros and ones, and count the labeled people in each population. Explain why population should be excluded from `X`. Now do the same with the challenge data, creating `X_challenge`, and report the shape of the dataframe.

```
import pandas as pd

label_data = pd.read_csv("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment3/planet
unknown_data = pd.read_csv("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment3/pla

X = label_data.drop(columns=["population"])
y = label_data[["population"]]

print(f"The shape of the feature matrix from the labeled data is {X.shape}.")
print(f"The shape of the label vector from the labeled data is {y.shape}.")
print(f"Checking to make sure all genotypes only have 0s and 1s... {X.iloc[:, 1:].isin([0, 1]).all().}")
print(y.value_counts())
```

```
X_challenge = unknown_data #no label should be in this csv
print(f"\nThe shape of the feature matrix from the unlabeled data is {X_challenge.shape}.")
```

The shape of the feature matrix from the labeled data is (500, 101).
The shape of the label vector from the labeled data is (500, 1).
Checking to make sure all genotypes only have 0s and 1s... True

```
population
pluto      100
saturn     100
mercury    100
venus      100
mars       100
Name: count, dtype: int64
```

The shape of the feature matrix from the unlabeled data is (50, 101).

Answer: Population should be excluded from `X` because that's the label that the model is trying to learn from the genotypes. If we included the population in `X`, then the model would just learn to identify the label by looking at the label column, defeating the purpose of training a model in the first place.

4b. Use `sklearn.decomposition.PCA(n_components=2)` to visualize the labeled genotypes. Plot one point per person, colored by population, with a legend. Label the axes with the percentage of variance explained by each component using `explained_variance_ratio_`. Use the original binary values; PCA automatically centers the columns. What fraction of the variance do the two components explain together? Which populations appear most similar, and which are clearly separated?

#Used ChatGPT to convert English instructions about how to plot the points per person different color

```
from sklearn.decomposition import PCA
```

```

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X.iloc[:, 1:])

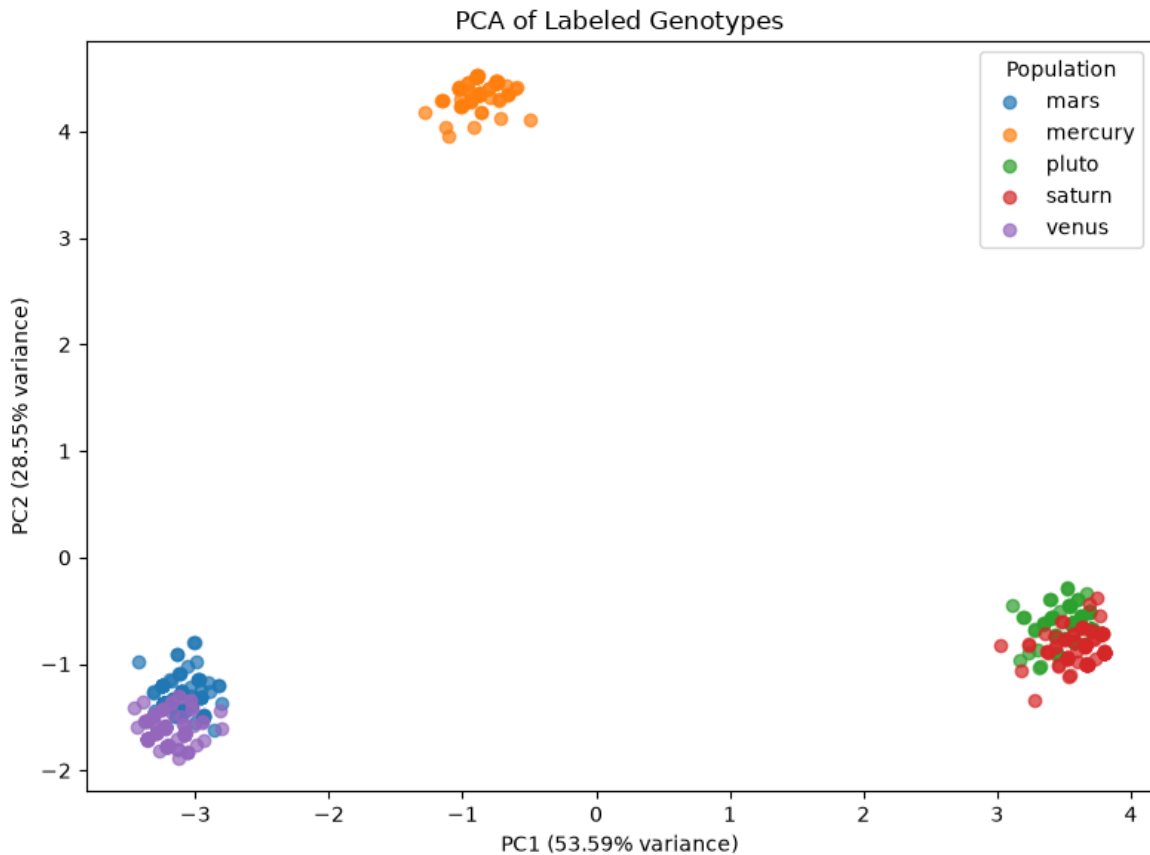
# Population labels
populations = y.values.ravel()

# Plot
plt.figure(figsize=(8, 6))

for population in sorted(set(populations)):
    mask = populations == population
    plt.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        label=population,
        alpha=0.7
    )

plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0] * 100:.2f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1] * 100:.2f}% variance)")
plt.legend(title="Population")
plt.title("PCA of Labeled Genotypes")
plt.tight_layout()
plt.show()

```



Answer: Together, the two components explain 82.14% of the variance. The populations of Venus and Mars appear similar and the populations of Saturn and Pluto appear similar. The Mercury population is clearly separated from the other two groups, and each of the groups consisting of population pairs is also separate from the other group.

4c. Use `sklearn.manifold.TSNE` with `n_components=2`, `perplexity=30`, `init="pca"`, `learning_rate="auto"`, and `random_state=2026`. Apply it to the original 100 variant columns and plot the result as above. Which populations appear most similar in this plot? Does this agree with PCA? Why should you be cautious about interpreting the distances between separate t-SNE clusters as genetic distances?

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, perplexity=30, init="pca", learning_rate="auto", random_state=2026)

X_tsne = tsne.fit_transform(X.iloc[:, 1:])

# Plot
plt.figure(figsize=(8, 6))

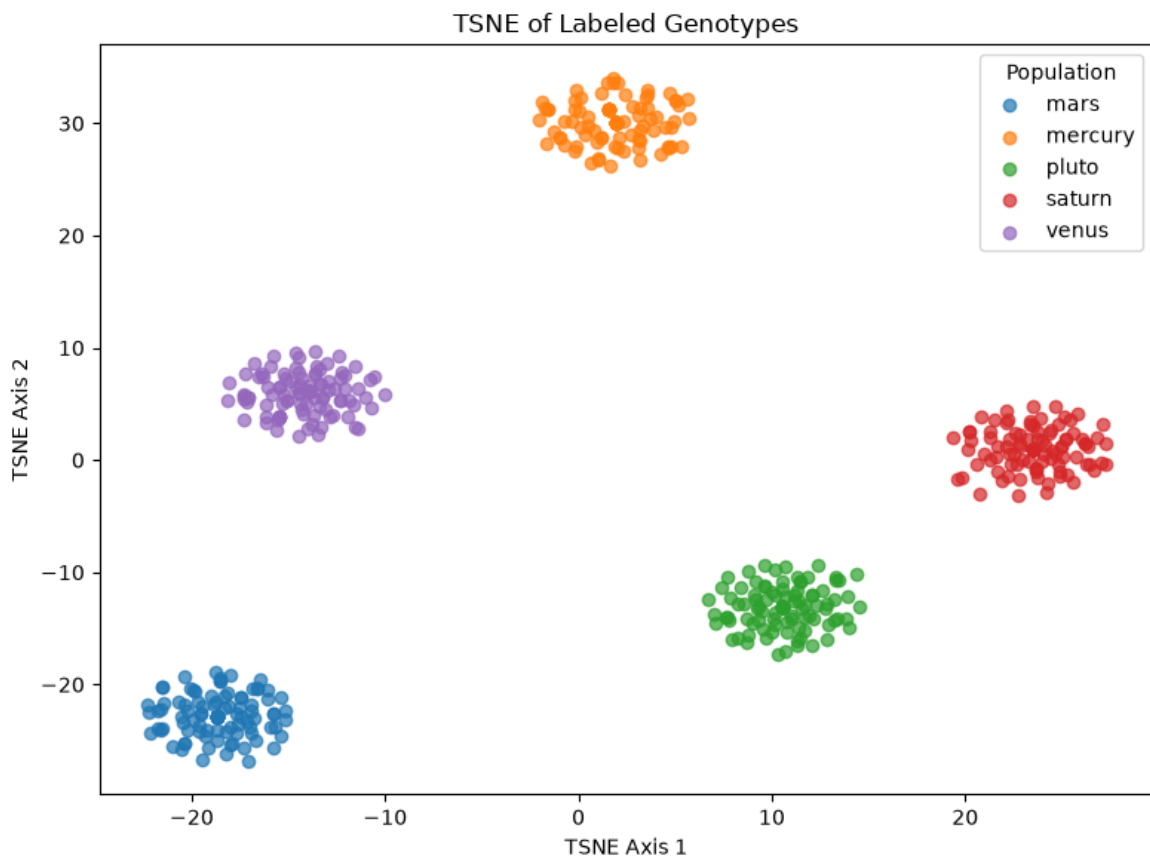
for population in sorted(set(populations)):
```

```

mask = populations == population
plt.scatter(
    X_tsne[mask, 0],
    X_tsne[mask, 1],
    label=population,
    alpha=0.7
)

plt.xlabel(f"TSNE Axis 1")
plt.ylabel(f"TSNE Axis 2")
plt.legend(title="Population")
plt.title("TSNE of Labeled Genotypes")
plt.tight_layout()
plt.show()

```



Answer: All of the populations appear to be well-separated in this plot. The two pairs from PCA are separated too, though they are closer to each other than any of the other groups. This agrees with PCA data. We shouldn't interpret T-SNE cluster distances as genetic distances since T-SNE is unstable, so different seeds will produce different results. In addition, the axes don't really carry any interpretable meaning; they're just optimized from the higher-dimensional data to display the

separation. Thus, the T-SNE cluster differences on the graph carry no significant biological meaning.

4d. Use `umap.UMAP` with `n_components=2`, `n_neighbors=15`, `min_dist=0.1`, `metric="hamming"`, `random_state=2026`, and `n_jobs=1`. Hamming distance is the fraction of variants at which two genotypes differ. Call `fit_transform(X)` without passing population labels, then use the labels to color the plot. Which populations appear most similar in UMAP? Compare your conclusions across all three plots, including any disagreement or uncertainty. Which method most clearly separates the populations? The first UMAP run may take longer while its numerical code compiles.

```
from umap import UMAP

umap = UMAP(n_components=2, n_neighbors=15, min_dist=0.1, metric="hamming", random_state=2026, n_jobs=1)

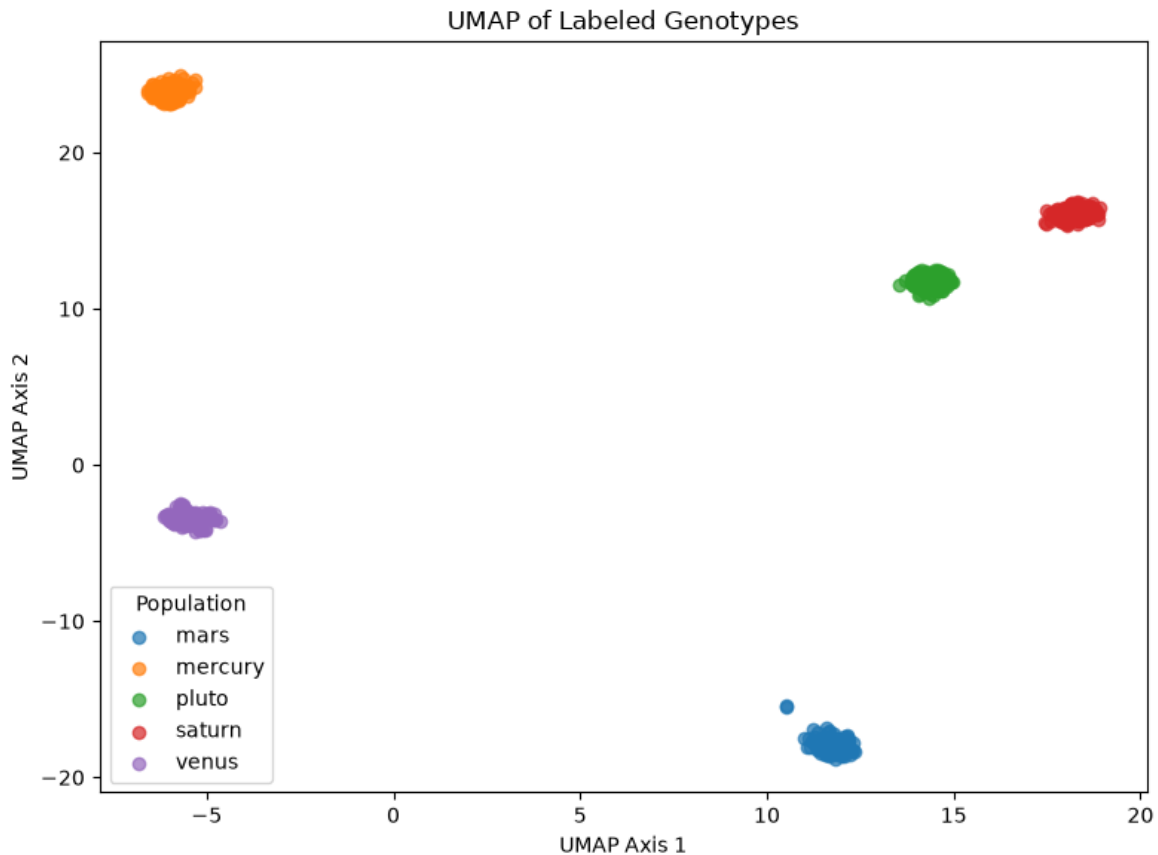
X_umap = umap.fit_transform(X.iloc[:, 1:])

# Plot
plt.figure(figsize=(8, 6))

for population in sorted(set(populations)):
    mask = populations == population
    plt.scatter(
        X_umap[mask, 0],
        X_umap[mask, 1],
        label=population,
        alpha=0.7
    )

plt.xlabel(f"UMAP Axis 1")
plt.ylabel(f"UMAP Axis 2")
plt.legend(title="Population")
plt.title("UMAP of Labeled Genotypes")
plt.tight_layout()
plt.show()

/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/assignment3/lib/python3.14/site-packages/umap/umap_.py:100: UserWarning: n_jobs=1 is deprecated, use n_jobs=-1 instead
  warn(
```



Answer: UMAP appears to perform similar to T-SNE in that it separates the populations well, keeping their area small and making distances between clusters large. Compared to T-SNE, the distances are even larger apart, emphasizing the differences between clusters. It agrees with T-SNE that the Saturn and Pluto populations are similar, but it places Mars and Venus populations far apart. Whereas PCA had them overlap and T-SNE had them closer than they were to Mercury, UMAP appears to show that the Venus population is more similar to the Mercury population than to that of Mars. UMAP most clearly separates the populations by maximizing spatial distance but both T-SNE and UMAP achieve perfect separation (as viewed by the human eye).

4e. Split the labeled data into 80% training and 20% validation using `sklearn.model_selection.train_test_split` with `stratify=y` and `random_state=2026`. Train a `sklearn.neighbors.KNeighborsClassifier` with `n_neighbors=5`, `metric="hamming"`, and `algorithm="brute"` on the 100 original variant columns. Report validation accuracy and plot a confusion matrix using `sklearn.metrics.ConfusionMatrixDisplay`. Fit the classifier only on the training split during validation. What accuracy would random guessing achieve? Based on your plots, which populations would you expect to be confused, and does the confusion matrix support that expectation? Does overlap in a 2D plot necessarily imply poor classification?

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix, accuracy_score
```


samples in the validation set, so we have a 1/5 chance for each randomly guessing it correct. This corresponds with an accuracy of 0.2. Based on the proximity of these two clusters in the T-SNE and UMAP plots, and the PCA data, I would expect the Pluto and Saturn populations to be confused. Also based on the PCA data, maybe the Venus and Mars populations can be confused too. However, since we're using K nearest neighbors, the proximity problem would likely be the more relevant one. The confusion matrix is inconclusive regarding that expectation since it performed perfectly across the validation data, so whether or not the model was most likely to confuse these two populations cannot be determined. It appears that it did a good job at classifying all populations, which could correlate to the high degree of separation between all populations in the T-SNE and UMAP plots. Not necessarily; overlap in a 2D plot could simply suggest the distances between two populations isn't great, but there could be other features in the populations that can contribute to a good classifier.

4f. After validation, refit the classifier on all 500 labeled people and predict the populations of the 50 challenge people. Save predictions.csv with exactly two columns, sample_id and predicted_population, in the original challenge file order and print into your solutions PDF. Keep each prediction attached to the correct sample ID. Make predictions from the genotypes without imposing a particular number of predictions per population.

```
knn = KNeighborsClassifier(n_neighbors=5, metric="hamming", algorithm="brute")
```

```
knn.fit(X.iloc[:, 1:], y)
```

```
y_challenge_pred = knn.predict(X_challenge.iloc[:, 1:])
```

```
output_df = pd.DataFrame({
    "sample_id": X_challenge["sample_id"],
    "predicted_population" : y_challenge_pred
})
```

```
output_df.to_csv("predictions.csv", index=False)
```

```
display(output_df)
```

```
/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/assignment3/lib/python3.14/site-packages/sklearn/neighbors/_base.py:100: UserWarning: X has 1 feature(s), but KNeighborsClassifier was expecting 2 features as input.
  return self._fit(X, y)
```

	sample_id	predicted_population
0	C_001	venus
1	C_002	saturn
2	C_003	mars
3	C_004	mercury
4	C_005	mercury
5	C_006	mercury
6	C_007	pluto
7	C_008	pluto
8	C_009	venus
9	C_010	saturn
10	C_011	saturn
11	C_012	venus
12	C_013	venus

13	C_014	mars
14	C_015	mercury
15	C_016	mercury
16	C_017	mercury
17	C_018	mars
18	C_019	venus
19	C_020	saturn
20	C_021	pluto
21	C_022	saturn
22	C_023	mars
23	C_024	mercury
24	C_025	venus
25	C_026	venus
26	C_027	saturn
27	C_028	mars
28	C_029	mars
29	C_030	saturn
30	C_031	pluto
31	C_032	venus
32	C_033	pluto
33	C_034	mars
34	C_035	mars
35	C_036	saturn
36	C_037	mercury
37	C_038	mars
38	C_039	pluto
39	C_040	saturn
40	C_041	mercury
41	C_042	pluto
42	C_043	mercury
43	C_044	saturn
44	C_045	pluto
45	C_046	mars
46	C_047	pluto
47	C_048	venus
48	C_049	venus
49	C_050	pluto